Oybek Sobirov's Write-up for COM 303 Final Project

The project was done in collaboration with Kymmani Allen

1. Introduction

BestBuy is one of the most well-known retail chains in the United States. Founded in 1966 in Minnesota under the name Sound of Music, the company has grown into the largest consumer electronics retailer in North America, with over 1,000 stores across the United States, Canada, and Mexico, approximately 90,000 employees, and around $43 billion in revenue as of 2024. It competes directly with major retailers like Amazon, Walmart, and Costco, and is particularly well known for its Geek Squad tech support service and its Totaltech membership program, which offers customers exclusive benefits and discounts in exchange for a yearly membership fee. What also makes BestBuy an interesting choice from a database perspective is that roughly 40% of its sales now come from online orders, which means the company operates both a large physical retail presence and a significant e-commerce platform at the same time — and both of these needed to be reflected in our design.

For our simulation, we modeled a simplified but realistic version of BestBuy's retail operations. We chose to focus on the sales side of the business, which meant tracking stores, products, vendors, customers, orders, and payments, while leaving out things like employee management and corporate finance, as the project instructions suggested. Our simulation includes five physical store locations spread across five different states — Los Angeles in California, New York in New York, Chicago in Illinois, Houston in Texas, and Seattle in Washington — as well as one online store representing bestbuy.com. Having stores in different states was an important decision for us, because it meant our state-level sales queries would actually return different and meaningful results rather than everything looking the same. We also made sure each store carries a slightly different selection of products, which reflects how a real BestBuy operates — not every product is available at every location, and a flagship store in New York is going to carry a different inventory mix than a smaller location in Seattle.

The products we carry in our simulation are split into four categories — Electronics, Appliances, Toys, and Furniture — which closely mirror BestBuy's actual department structure. In total we have 20 products supplied by seven real-world vendors: Apple, Samsung, Sony, LG, Dyson, Mattel, and IKEA. Each product is linked directly to its vendor in the database, which allows us to run vendor-level analysis and answer questions like which vendor is responsible for the most units sold. Prices in our simulation are fixed per product rather than varying by store for the sake of simplification given the scope of the project, though in a real-world system pricing would likely differ by location.

For customers, we followed BestBuy's membership model and split them into two types — Members and Guests. Members are customers who have signed up with their name and email address, similar to

BestBuy's Totaltech program, and they are tracked across purchases. Guests are completely anonymous, meaning we record that a purchase happened but we do not store any identifying information about who made it. This distinction turned out to be one of the more important business rules in our design, particularly when it came to the online store. The project specification was clear that the website should have no anonymous customers, so we made it a rule that anyone placing an order through bestbuy.com — which we modeled as a special store with storeID 6 and no physical address — would be required to create a member account first. This rule could not be enforced at the schema level alone, so we handled it in our Python interface, which I will describe in more detail in the interfaces section.

In terms of the order process, customers can browse the inventory at their chosen store, add items to a cart, and check out with one of four payment methods — credit card, debit card, PayPal, or gift card. One thing we wanted to make sure our simulation captured correctly was tax calculation, since BestBuy customers in different states pay different sales tax rates. We applied real state tax rates at checkout — 7.25% for California, 8% for New York, 6.25% for both Illinois and Texas, and 6.5% for Washington — while orders placed through the online store are not taxed, since BestBuy.com does not have a physical presence in every state. This was handled automatically in our checkout interface based on which store the order was placed at.

One business decision that is worth mentioning here is how we handled returns. Our original design included a separate Return table that would track which order items had been returned. However, as we got further into the implementation, we ran into a structural problem with it, which I will explain in more detail in the problems section. After discussing it with our professor, we made the decision to drop the Return table entirely and instead handle returns at the payment level — a return would simply be recorded as a new order with a refund payment, which is actually consistent with how many large retailers handle returns in their databases. This ended up being a cleaner and more realistic approach than what we had originally planned, and it kept our schema simpler without losing any meaningful functionality for the scope of this project.

2. Significant Problems and Solutions

When we first started working on the ER diagram, we ran into several problems that we had to think through carefully before we could move forward with the implementation. Some of them were design decisions we made early on, and some of them only became obvious once we actually started building the tables in MySQL.

The first major thing we had to figure out was how to handle the three specialization hierarchies in our design — Store, Product, and Customer. The issue was that if we tried to put everything into a single table for each of these, we would end up with a lot of null values. For example, if we had only one Store table with both a URL column and a Location column, every Brick and Mortar store would have a null

URL, and the online store would have a null address, city, and state. That kind of design is messy, and it creates problems when you try to query the data. So we decided to use specialization hierarchies for all three, where each supertype table holds only the shared attributes, and each subtype table holds only what is specific to that type. This kept the tables clean and made sure there were no unnecessary nulls anywhere in the schema.

The second major problem we ran into was cardinality — specifically, how to handle many-to-many relationships between certain tables. When we first drew the ER diagram, it became clear pretty quickly that a product can appear in many orders, and an order can contain many products. The same thing applies to products and stores through inventory — a product can be at many stores, and a store can carry many products. You cannot represent a many-to-many relationship directly in a relational database without causing serious problems like data redundancy, insert anomalies, delete anomalies, and update anomalies. To solve this, we created three associative entities — Order_Item, Order_Payment, and Inventory. These are tables that sit between the two related tables and resolve the many-to-many relationship by taking the primary keys of both sides as foreign keys and combining them into a composite primary key. For example, Order_Item takes orderID from Orders and productID from Product and makes those two together the primary key of the table. This also has the additional benefit of keeping everything in Third Normal Form, since associative entities by definition only store data that exists because of the relationship between the two tables they connect — there is no data in Order_Item that belongs to Order alone or to Product alone.

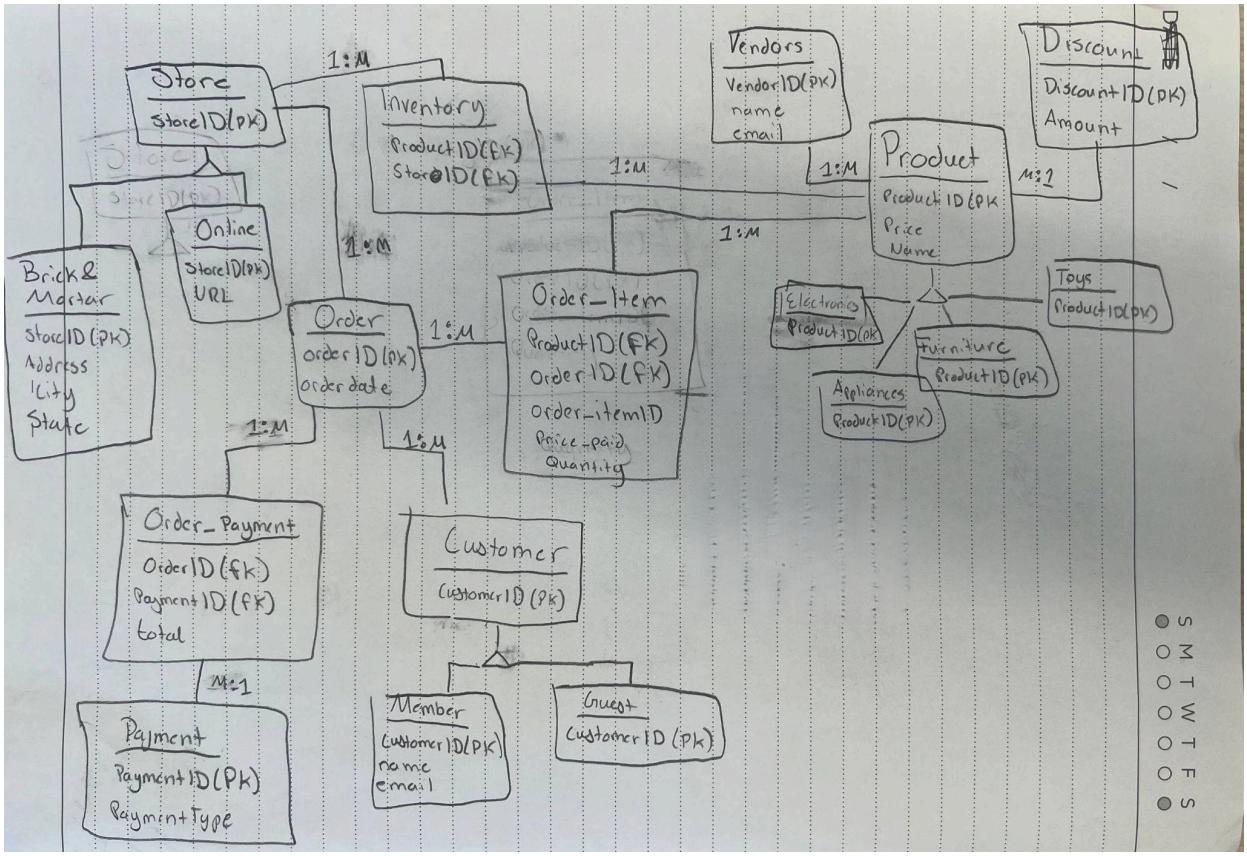
The third problem was with the Return table. Our original design included a Return table that was supposed to track which items had been returned by customers. The problem we ran into was that a return does not just affect one table — when a return happens, the payment total needs to be updated, the inventory might need to be updated, and the original order record is involved too. We initially tried to design a Return table that referenced order_item, but order_item uses a composite primary key made up of orderID and productID together. That means there is no single column in order_item that we could reference as a foreign key in the Return table, and adding a surrogate key just to fix this would have required us to restructure the entire order_item table. We talked about it with our professor, who suggested that simpler is better, and we agreed. We decided to drop the Return table entirely and instead handle returns as a function at the application level — a return would update the total attribute in order_payment to reflect the refund, which actually mirrors how a lot of real retailers handle returns in their systems anyway.

Finally, one design decision we made early on that saved us a lot of trouble later was how we handled the online store. The project specification said the website should have no anonymous customers — meaning no guests. We realized quickly that there was no way to enforce this rule at the schema level alone, since the database itself does not know whether an order is coming from the website or from a physical store

without some additional logic. So we handled it at the application level in our Python interface, where we check the storeID before allowing checkout — if the storeID is 6, which is our online store, the system requires the customer to create a member account before the order can go through.

3. ER Diagram

We started the ER diagram by hand, which you can see in the image below. We sketched it out on paper first so we could think through the relationships before committing to anything in the actual database.



The diagram is organized around three specialization hierarchies, three associative entities, and several supporting tables. Here is a breakdown of how everything fits together.

Store Hierarchy

The Store table is the supertype and holds only storeID as its primary key. It has two subtypes — BM (Brick and Mortar) and Webstore. BM stores carry address, city, and state as their specific attributes, while the Webstore carries a URL. We have five BM stores and one Webstore in our simulation. The reason we split them this way rather than keeping everything in one table is the null problem described

earlier — a Webstore has no physical address, and a BM store has no URL, so combining them into one table would have meant half the columns being null for every row.

Customer Hierarchy

The Customer table is the supertype with just customerID as its primary key. It has two subtypes — Member, which adds name and email, and Guest, which has no additional attributes since guests are anonymous by definition. The split here reflects BestBuy's real membership model, and it also enforces our business rule that online customers must be members.

Associative Entities

Inventory sits between Store and Product and resolves the many-to-many relationship between them. Its composite primary key is made up of productID and storeID. This table tells us which products are available at which stores, and since not every product is at every store, the rows in this table are intentionally sparse.

Order_Item sits between Orders and Product and resolves the many-to-many relationship between them. Its composite primary key is orderID and productID. It also stores quantity, which is data that only makes sense when a specific product meets a specific order — it does not belong in the Orders table or the Product table alone, which is exactly what makes it a good associative entity.

Order_Payment sits between Orders and Payment and resolves the many-to-many relationship there — a single order could theoretically be split across multiple payment methods. Its composite primary key is orderID and paymentID, and it also stores the total amount paid.

Orders and the Transaction Chain

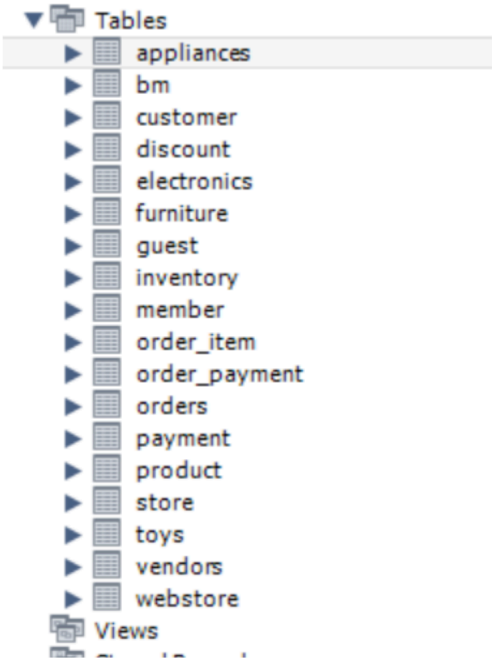
The Orders table is the central table in the transaction chain. It holds orderID, orderDate, storeID, and customerID as foreign keys linking each order to both the store where it was placed and the customer who placed it. From Orders, you can reach Order_Item to see what was purchased, and Order_Payment to see how much was paid and by what method.

The complete transaction chain — Orders to Order_Item to Product, and Orders to Order_Payment to Payment — captures everything we need to run the required business queries, from top-selling products to store-level revenue to market basket analysis.

4. Relational Schema

After finishing the ER diagram, we translated it into a relational schema and implemented it in MySQL. The screenshot below shows all 18 tables as they appear in MySQL Workbench, along with the DDL for

some of the core tables.




```
create table product(
    productID int primary key,
    vendorID int,
    discountID int,
    price decimal(10,2),
    name varchar(50),
    foreign key(vendorID) references vendors(vendorID),
    foreign key(discountID) references discount(discountID)
);

create table orders(
    orderID int primary key,
    orderDate date,
    storeID int,
    customerID int,
    foreign key(storeID) references store(storeID),
    foreign key(customerID) references customer(customerID)
);

create table order_item(
    orderID int,
    productID int,
    quantity int,
    primary key(orderID,productID),
    foreign key(orderID) references orders(orderID),
    foreign key(productID) references product(productID)
```

The schema follows directly from the ER diagram, with a few refinements we made along the way based on normalization principles. Every table has a clearly defined primary key, all foreign key constraints are explicitly declared, and the overall design is in Third Normal Form — meaning there is no redundant data, no transitive dependencies, and every non-key attribute depends only on the primary key of its table.

Here is a description of each table and what it is responsible for:

store holds only the storeID as its primary key. It exists as the parent table for both BM and webstore and is what both of those tables reference through their foreign keys.

BM represents physical store locations. It inherits storeID from store as both its primary key and its foreign key, and adds address, city, and state. We have five rows in this table, one for each of our physical store locations across California, New York, Illinois, Texas, and Washington.

webstore represents the online store. It also inherits storeID from store, and adds a URL column. There is only one row in this table since we only have one online store, which is bestbuy.com with storeID 6.

vendors holds vendorID, name, and email. We have seven vendors in our simulation — Apple, Samsung, Sony, LG, Dyson, Mattel, and IKEA. Every product in our database is linked to one of these vendors.

discount holds discountID and amount. Discounts are optional and are linked to products through a foreign key in the product table. Not every product has a discount applied to it.

product is one of the central tables in the schema. It holds productID as its primary key, along with vendorID and discountID as foreign keys, and price and name as its own attributes. This table is the parent for all four product subtype tables.

electronics, **appliances**, **toys**, and **furniture** are the four product subtype tables. Each one takes productID as both its primary key and its foreign key referencing product, and each adds one subtype-specific attribute — watts for electronics, brand for appliances, age for toys, and size for furniture.

inventory is an associative entity that sits between store and product. Its composite primary key is made up of productID and storeID, both of which are also foreign keys. This table tracks which products are available at which stores, and since we made inventory intentionally sparse, not every product appears at every store.

customer holds only customerID as its primary key. It is the parent table for both member and guest.

member inherits customerID from customer as its primary key and foreign key, and adds name and email. These are the customers who have signed up for an account.

guest inherits customerID from customer as its primary key and foreign key, and has no additional attributes. These are anonymous customers who chose not to create an account.

payment holds paymentID and payment_type. We have four payment types in our simulation — credit card, debit card, PayPal, and gift card.

orders holds orderID as its primary key, along with orderDate, storeID, and customerID as foreign keys. This is the central table in the transaction chain and connects every order to the store where it was placed and the customer who placed it. We named this table "orders" rather than "order" because ORDER is a reserved word in MySQL.

order_item is an associative entity between orders and product. Its composite primary key is made up of orderID and productID, both of which are foreign keys. It also stores quantity, which is data that only exists when a specific product is part of a specific order.

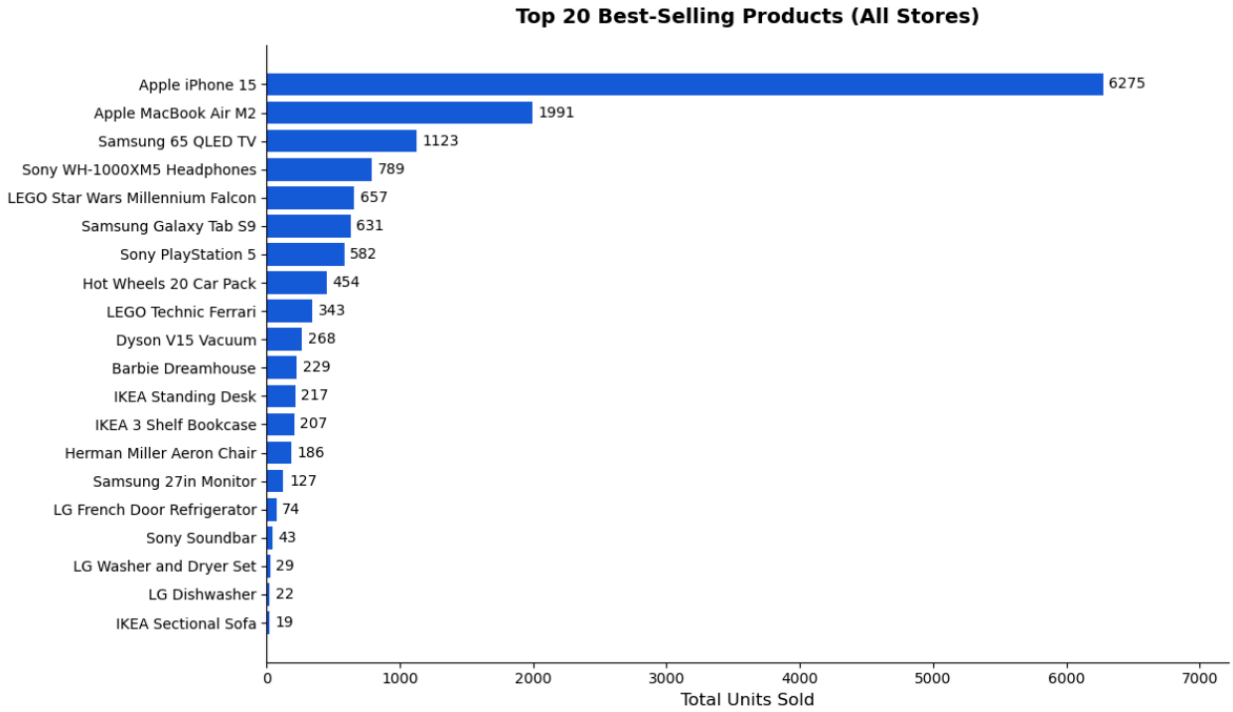
order_payment is an associative entity between orders and payment. Its composite primary key is made up of orderID and paymentID, both foreign keys. It also stores total, which is the final amount charged for that order including tax.

One thing worth noting about this schema is that price is stored only in the product table and is not duplicated anywhere else. Our original design had a price_paid column in order_item, but we removed it during the normalization process because it was redundant — the price already exists in product, and storing it again in order_item would have violated Third Normal Form by introducing data that could become inconsistent if a product's price ever changed.

5. Sample Queries (Subject to change as we are testing the interface + testing placing orders)

For our queries we ran nine total — five required by the project instructions and four additional ones that we included in the write-up for a more complete picture of what the database can tell us. All queries were run directly in MySQL Workbench and also through our Python interface in Jupyter Notebook, where we generated charts from the results using pandas and matplotlib.

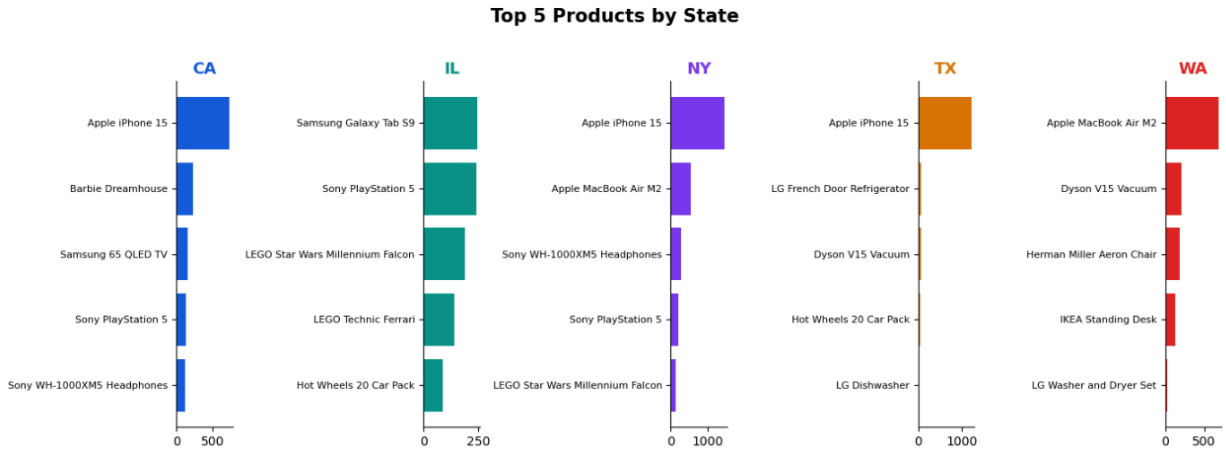
Query 1 — Top 20 Best-Selling Products Across All Stores



This query joins order_item with product and groups by productID to get the total quantity sold for each product across all stores and all orders. We sorted the results in descending order and limited to the top 20. The result shows Apple iPhone 15 as the clear top seller at 6,275 units, followed by Apple MacBook Air M2 at 1,991 and Samsung 65" QLED TV at 1,123. Appliances like the LG Washer and LG Dishwasher sit at the very bottom, which makes sense given that people do not buy appliances frequently. From a

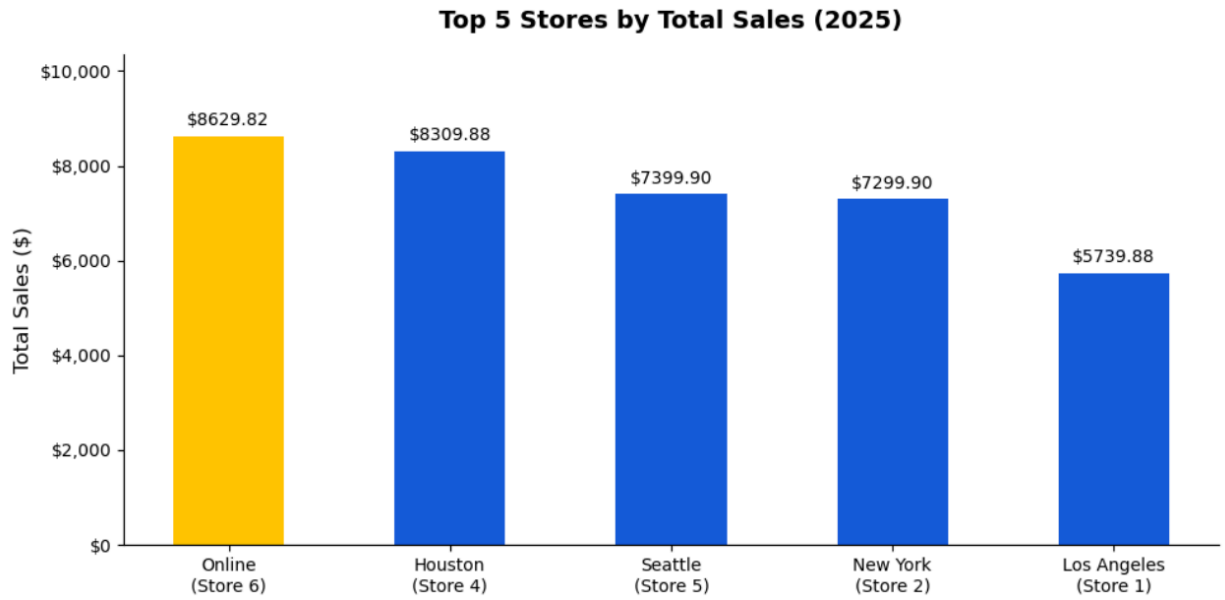
business perspective, this query is something a corporate manager at BestBuy would run regularly to decide which products to prioritize for restocking and which vendors to negotiate better pricing with.

Query 2 — Top 5 Products in Each State



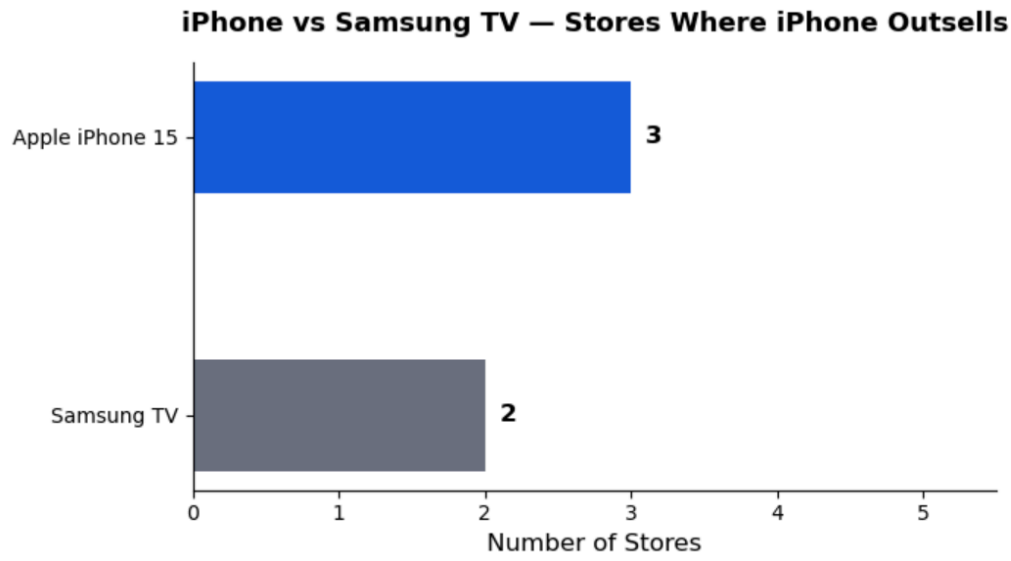
This query is similar to the first but adds a JOIN to the BM table so we can filter by state and group results at the regional level. Because we have stores in five different states, the results show meaningfully different buying patterns across locations. California is driven by iPhone and Barbie Dreamhouse, Illinois is dominated by Samsung Galaxy Tab and PS5, and Washington's top seller is MacBook rather than iPhone. This kind of regional breakdown is exactly what a regional manager would use to make inventory and promotion decisions specific to their market — what sells in Chicago is not necessarily what sells in Seattle.

Query 3 — Top 5 Stores by Total Sales in 2025



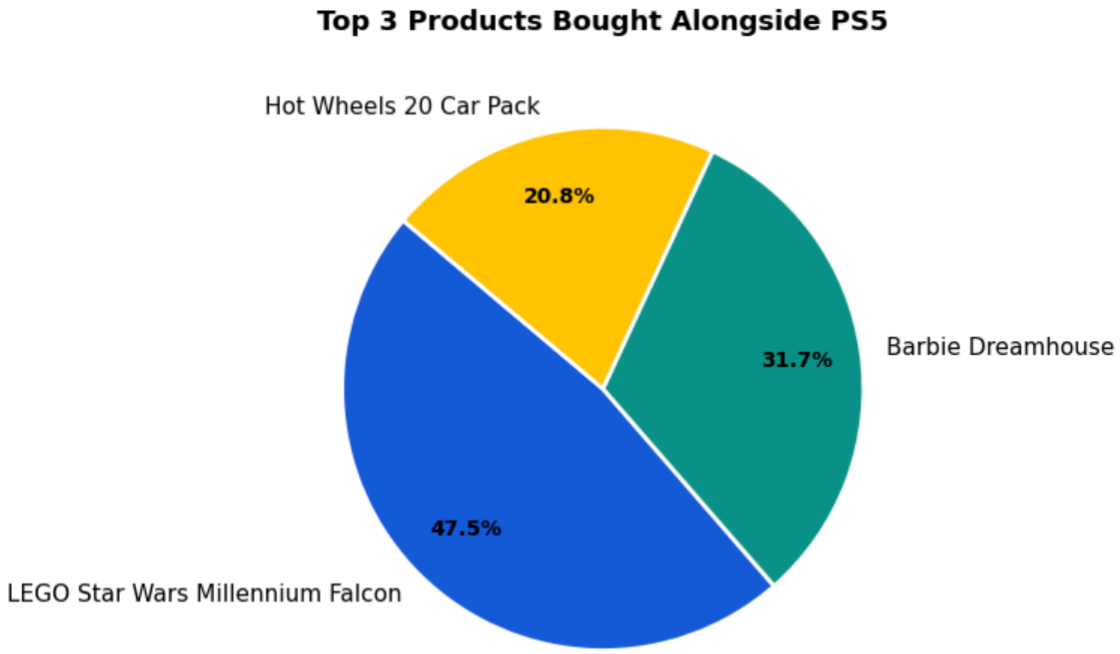
This query joins orders with order_payment and groups by storeID to get the total revenue generated by each store. We filtered by year to show only 2025 data and limited the results to the top five. The online store came in first at $8,629.82, followed closely by Houston at $8,309.88 and Seattle at $7,399.90. Los Angeles came in last at $5,739.88. The fact that the online store leads in total revenue is consistent with real BestBuy trends where e-commerce has been growing significantly. This query is something an executive team would use to evaluate store performance and decide where to invest resources.

Query 4 — In How Many Stores Does iPhone Outsell Samsung TV



This is our version of the Coke vs Pepsi query from the project instructions, adapted for BestBuy's product catalog. We used a subquery to find all stores where the total quantity of iPhone 15 sold exceeded the total quantity of Samsung 65" QLED TV sold, then counted the distinct stores. The result was that iPhone outsells Samsung TV in 3 out of 5 stores. This kind of brand comparison query has direct business value — if Apple is outselling Samsung in the majority of your locations, it affects how you negotiate with both vendors, how you allocate shelf space, and where you focus your marketing budget.

Query 5 — Top 3 Products Bought Alongside PS5

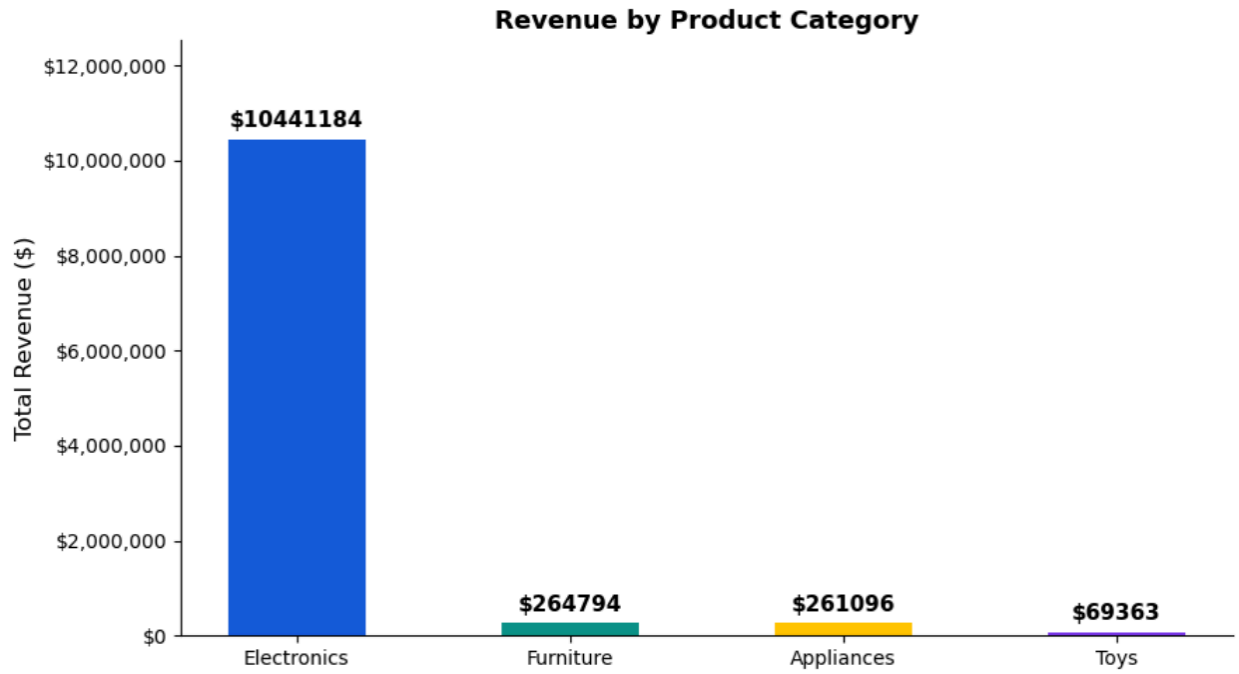


This is our market basket analysis query, which is the equivalent of the "what do customers buy alongside milk" question from the project instructions. We used a subquery to find all orders that contained a PS5, then looked at what other products appeared in those same orders. The results showed that LEGO Star Wars Millennium Falcon was bought alongside PS5 in 47.5% of cases, followed by Barbie Dreamhouse at 31.7% and Hot Wheels at 20.8%. This is an interesting finding because it suggests that PS5 buyers are often parents shopping for multiple children in a single trip. The practical application of this query is bundle promotions and store layout — placing LEGO and toy displays near the gaming section could drive additional purchases.

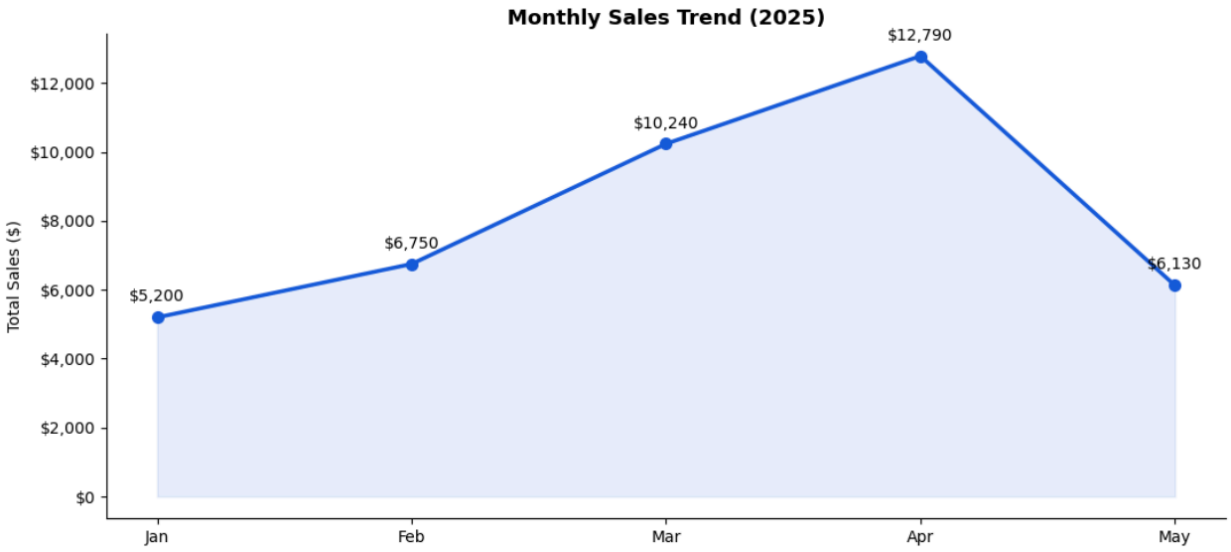
Additional Queries

Beyond the five required queries, we ran four more that we felt gave a more complete picture of the business.

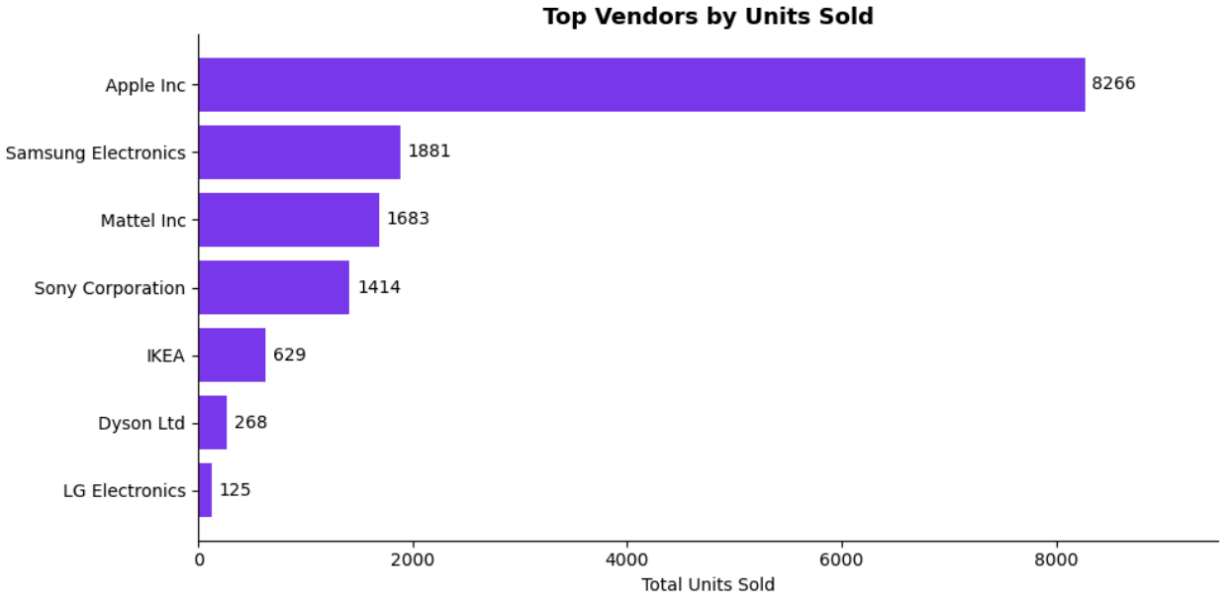
Query 6 looked at total revenue broken down by product category. The results showed that Electronics generated approximately $10.4 million in revenue, while Furniture came in at $264,000, Appliances at $261,000, and Toys at $69,000. The massive gap between Electronics and everything else reflects the combination of high unit prices and high sales volume that electronics products carry in our dataset.



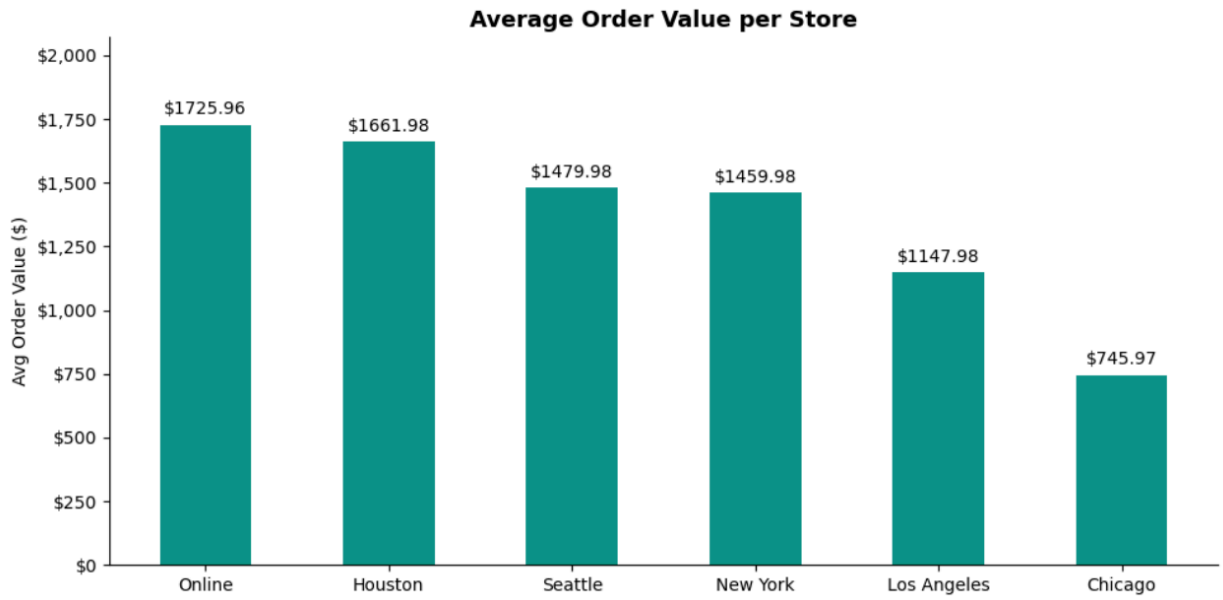
Query 7 tracked monthly sales across 2025. We saw a steady upward trend from January through April, with April peaking at $12,790, before dropping in May. The April spike aligns with real-world tax refund season in the United States, when consumer electronics spending tends to increase significantly.



Query 8 ranked vendors by total units sold. Apple led by a large margin at 8,266 units, followed by Samsung at 1,881 and Mattel at 1,683. LG came in last despite having multiple appliance products in our catalog, reflecting the low unit volumes that appliances naturally generate.



Query 9 looked at average order value per store. The online store had the highest average at $1,725.96 per order, while Chicago had the lowest at $745.97. This metric is more nuanced than total sales alone because a store with fewer orders but a higher average order value might actually be performing better in terms of customer quality than a high-volume but low-average-value location.



The four additional queries were not included in the demo portion of our presentation because they go beyond what the project instructions specifically asked for. We felt they added value to the write-up as a demonstration of what else the database can support, but including all nine in the demo would have made the presentation too long given the 15-minute time limit.

6. Test Data and Interfaces

For our test data, we manually wrote INSERT statements to populate all 18 tables. We ended up with 6 stores, 20 products, 7 vendors, 15+ customers, 30+ orders, and over 60+ order items spread across January through May 2025. One thing we were intentional about was making the quantities realistic rather than just random. iPhone and MacBook have significantly higher quantities than anything else because that reflects how BestBuy actually sells — consumer electronics like phones and laptops move in much higher volumes than a refrigerator or a sofa. Appliances have low quantities on purpose for the same reason. We also made sure inventory was sparse, meaning not every product is available at every store, because if every store had every product the per-store and per-state queries would return identical results and there would be nothing interesting to compare. Having stores across five different states was also important for the same reason — it made the regional queries actually meaningful.

For the interfaces, we built everything in Jupyter Notebook using Python with the mysql-connector-python library, pandas, and matplotlib. The main menu has six options. The first is Place an Order, which walks through the full checkout flow — the customer picks a store, browses the inventory at that store, adds items to a cart, sees an order summary with tax calculated based on the store's state, chooses to register as a member or check out as a guest, selects a payment method, and the order gets written to the database. Online orders require a member account — guests are not allowed, which mirrors the project

specification. The second option runs all five required analytics queries and generates live charts from the database. The remaining options cover viewing inventory at a store, adding a product to a store's inventory, looking up a customer by ID, and viewing order history for a customer. The packages needed beyond the standard installation are mysql-connector-python, pandas, and matplotlib.

7. Summary

Overall we are happy with what we were able to put together for this project. We started with a hand-drawn ER diagram on paper, translated it into 18 normalized MySQL tables, populated it with realistic data, ran nine business queries with live charts, and built a working Python interface that handles real checkout flows including tax, membership logic, and database writes. Looking back at it, the project covered the full database development lifecycle — conceptual design, logical design, implementation, data population, querying, and application building — which is something that is hard to appreciate until you actually go through all of it from start to finish. The schema ended up clean and well-structured, the queries return meaningful results, and the interface does everything it needs to do.

8. Reflection

Honestly, this project was more work than we expected going in, but it was also more rewarding than we expected. The part that surprised us the most was how much of the real difficulty is in the design phase rather than the coding phase. Writing SQL queries is something we had practice with from class, but actually sitting down and deciding how to structure 18 tables so that they work together cleanly, enforce the right constraints, and support all the queries you need — that is a different kind of thinking. We had to go back and revise the ER diagram more than once after we started building the tables and realized certain things were not going to work the way we had drawn them.

Working as a pair was helpful because we could divide the work — one of us focused on the DDL and schema while the other worked on data population and queries — but it also meant we had to stay on the same page about design decisions, which required more communication than we initially planned for. The Return table situation is a good example of that — it took some back and forth between us and with the professor before we landed on the right solution.

Going into the project we thought the database part was the main challenge, but building something that actually writes to the database in real time, calculates tax by state, enforces the online guest rule, and generates live charts from query results made it feel like a real working system rather than just a school assignment. If we were to do it again, we would probably plan the application layer earlier in the process rather than treating it as something to figure out at the end, because some of the interface requirements actually revealed things about the schema that we had to go back and fix. That was a good lesson about

how database design and application design are not as separate as they might seem when you first start learning about them.